MODULE 20 · DOMAIN 2 - IMPLEMENT GENERATIVE AI AND AGENTIC SOLUTIONS

Agents IV: autonomy with safeguards

Levels of autonomy, risk-tiered approvals, identity without a user, monitoring deployed agents, evaluating behaviour, and error analysis.

DOMAIN 2 - Generative AI and agents WEIGHT 30-35% MODULE 20 of 39

admin@sthithapragna.com

Autonomy is **earned per door**. The intern opens low-risk doors alone, and needs a **manager's signature** for the rest.

How independently does the agent act?

LEVEL	HOW IT WORKS	IDENTITY IT ACTS WITH
Assistive	Acts for a user who is present	The user's, delegated
Semi-autonomous	Works alone; pauses for approvals	Varies; people approve
Autonomous	Background, on events or schedules	The agent's own identity
Autopilot (preview)	Works like a colleague in Microsoft 365	Entra Agent ID, Agent 365

ON THE EXAM

Most real designs are semi-autonomous: autonomy with approval gates.

Match the control to the consequence

ACTION	EXAMPLES	CONTROL
Read-only	Search, lookups, calculations	Run freely; trace it
Reversible writes	Draft a document, create a ticket	Run, log, and monitor
Consequential or irreversible	Payments, deletions, external emails	Require approval first
Never allowed	Anything outside the agent's job	No tool for it at all

ON THE EXAM

The strongest control for a forbidden action isn't approval. It's not giving the agent that tool.

Scope approval to the **tool call**, not the whole agent. Low-risk steps **flow**; high-risk steps **wait**.

admin@sthithapragna.com

How approvals actually work

- **MCP tools:** require_approval is on by default. The response contains an **approval request**.
- Your app shows it to a person, then sends back an **approval response**: approve or deny.
- **Agent Framework:** approval-required tools **pause the workflow** until someone answers.
- At mandatory gates, **checkpoint the state**, so the run resumes without replaying earlier work.
- Decide whether a human gives an **approval that advances**, or **feedback that loops back**.

> ON THE EXAM
>
> Approvals can take hours. Checkpoints stop a long wait from throwing away completed work.

Handling an MCP approval request

```
response = openai.responses.create(extra_body=agent_ref, input=request)

for item in response.output:
    if item.type == "mcp_approval_request":
        approved = ask_reviewer(item.server_label, item.name, item.arguments)
        response = openai.responses.create(
            extra_body=agent_ref,
            previous_response_id=response.id,
            input=[{"type": "mcp_approval_response",
                    "approval_request_id": item.id,
                    "approve": approved}])
```

ON THE EXAM

The reviewer sees which server, which tool, and exactly which arguments, before anything happens.

Safeguards for agents that act alone

- **Least-privilege identity** and **allowed tools only**: the keycard from module 17.
- **Validation in the tool's code**, so limits hold whatever the agent sends.
- **Turn and iteration limits**, and termination conditions on every loop.
- **Guardrails at tool calls and tool responses** (preview), not only at user input and output.
- **Versioned releases**, so a misbehaving version can be rolled back quickly.

> ON THE EXAM
>
> Nobody is watching an autonomous run. Every safeguard has to work without a person noticing the problem first.

Whose permissions does the agent use?

A USER IS PRESENT

On-Behalf-Of

- The user's token is available
- The agent acts with the user's delegated permissions
- It can't exceed what that user may do

NO USER (AUTONOMOUS)

The agent's own identity

- No user token exists
- The agent uses its own Entra agent identity
- Default access: models and session storage
- Other resources need roles assigned to it

THE SIGNAL

A background agent failing with 403 on your storage account usually lacks a role on its own identity.

Knowing which agent did what

- Agents in development **share the project's agent identity**.
- **Publishing** gives an agent its **own unique identity**, so its actions are attributable.
- Grant roles to the published identity, never broadly to the shared one.
- Autopilot agents (preview) add an **Entra Agent ID**, governed through **Agent 365**.
- Foundry Control Plane shows the whole agent fleet, as module 10 showed.

> ON THE EXAM
>
> An autonomous agent that acts under a shared identity can't be audited separately from its neighbours.

The Agent Monitoring Dashboard

- Open an agent under **Build**, then its **Monitor** tab.
- **Operational:** token usage, latency and success rates.
- **Evaluation:** scores from continuous evaluation of sampled production traffic.
- **Security:** results from scheduled red-teaming scans. A failed scan means remediation.
- The **Monitor settings** panel chooses which evaluations and security checks run.

> ON THE EXAM
>
> The dashboard reads the project's Application Insights. No App Insights, no dashboard.

Keeping watch after release

SIGNAL	WHAT IT CATCHES
Continuous evaluation	Quality and safety problems in live, sampled traffic
Scheduled evaluation	Drift, measured against a fixed dataset
Scheduled red teaming	New weaknesses against disguised attacks
Azure Monitor log alerts	Thresholds crossed, such as tool failure rate
Traces linked to results	The exact run behind any low score

ON THE EXAM

A log alert on a KQL query, routed through an action group, turns a dashboard number into a page to someone on call.

Did the agent do the right things?

- **Intent Resolution:** did it understand the request?
- **Tool Call Accuracy**, and the narrower tool evaluators: right tool, right arguments?
- **Task Adherence** and **Task Completion:** did it follow instructions and finish?
- **Prohibited Actions** and **Sensitive Data Leakage:** did it cross a line?
- **Compare runs** before and after each change, to prove fixes and catch regressions.

> ON THE EXAM
>
> Autonomous agents need action-focused evaluators. Fluent text can hide a wrong tool call.

Cluster analysis

- Groups evaluation results with **similar problems**: cluster, then subcluster, then entry.
- For example, inadequate final answers, caused by an invalid or missing API key.
- Each subcluster shows its entry count, statistics and **recommendations**.
- Filter by score, evaluator or time, and **download as CSV**.
- The analysis **isn't stored**. Leave the page, and it's gone.

> ON THE EXAM
>
> Fix the biggest cluster first. One root cause often explains hundreds of low scores.

The error analysis loop

01
Evaluate
Agent evaluators on a dataset or traces.

02
Cluster
Group failures by shared cause.

03
Inspect
Open traces for sample entries.

04
Fix
Instructions, tools, schemas or data.

05
Re-evaluate
Run the same evaluation again.

06
Compare
Before and after, side by side.

ON THE EXAM

Microsoft's agent evaluation loop: identify weak areas, cluster, adjust, then re-evaluate and compare.

Agent symptoms and likely causes

LOW SCORE OR SYMPTOM	LOOK FIRST AT
Tool Call Accuracy	Tool names, descriptions and schemas
Intent Resolution	Ambiguous requests, or unclear instructions
Task Adherence	Conflicting instructions, or injected content
Tool Call Success	Authentication, endpoints, network access
Task Completion, with good turns	The overall flow, handoffs and turn limits
Rising cost and latency	Tool loops and growing context

ON THE EXAM

Evaluators tell you what failed. Traces tell you how. Clusters tell you how often.

> A procurement agent researches suppliers with web search and Azure AI Search, then places orders through an MCP tool called place_order.
>
> Policy: a buyer must approve every order. Research must run without interrupting anyone.

How should the team configure approvals?

A Require approval only for place_order; let research tools run freely.

B Require a person to approve every tool call the agent makes.

C Tell the agent in its instructions to ask a buyer before ordering.

D Set tool_choice to none whenever the agent is about to place an order.

◆ PAUSE AND DECIDE

How should the team configure approvals?

A Approval on place_order only.

Gates the consequential action, and leaves read-only research autonomous.

B Approve every tool call.

Interrupts research constantly, breaking the second half of the policy.

C Instruction to ask first.

A job description, not a gate. Nothing stops an unapproved order.

D tool_choice none before ordering.

The app can't know in advance, and it would block orders, not route them to approval.

THE TIEBREAKER

Approve the tool call that has consequences. Everything else flows.

> A published agent runs every night with no user signed in. It summarises the day's support tickets and writes a report to the company's Azure Storage account.
>
> Model calls succeed, but every write to the storage account fails with 403 Forbidden.

What should the team do?

A Use On-Behalf-Of so the agent acts with a signed-in user's permissions.

B Put the storage account's access key into the agent's instructions.

C Assign a storage data role to the agent's own Entra agent identity.

D Give every employee the Foundry User role on the agent's project.

◆ PAUSE AND DECIDE

What should the team do?

A On-Behalf-Of.
Needs a user token, and nobody is signed in at night.

B Access key in instructions.
A secret in the least protected place, visible in traces and logs.

C Role for the agent identity.
External resources need roles assigned to the agent's own identity.

D Foundry User for everyone.
Project access for people, which does nothing for the agent's storage writes.

THE SIGNAL

Models work, storage doesn't, and no user is present: the agent identity lacks a role on that resource.

> An evaluation of a support agent produced 600 failing samples across several agent evaluators.
>
> Two engineers are reviewing failures one at a time, and after a week have no clear picture of what to fix first.

What should the team do?

A Double the evaluation dataset so the failure rate becomes more precise.

B Run cluster analysis, fix the largest cluster, then re-run and compare.

C Lower each evaluator's pass threshold until most samples pass.

D Remove the failing samples from the dataset before the next run.

◆ PAUSE AND DECIDE

What should the team do?

A Double the dataset.
More failures to review, and still no grouping by cause.

B Cluster, fix largest, compare.
Groups failures by shared cause, prioritises impact, and proves the fix.

C Lower thresholds.
Hides the failures. The agent behaves exactly as badly.

D Remove failing samples.
Deletes the evidence, and the next evaluation looks falsely healthy.

THE SIGNAL

'Too many failures to review one by one' is the job cluster analysis was built for.

> An autonomous invoicing agent runs around the clock. Its tool calls are traced to the project's Application Insights.
>
> The operations team wants to be notified within minutes if the agent's tool failure rate rises above 5 percent.

What should the team set up?

A. A daily manual check of the Agent Monitoring Dashboard by one engineer.

B. A weekly scheduled evaluation against a fixed test dataset.

C. Agent instructions telling it to email operations when tools fail.

D. An Azure Monitor log alert on a KQL query, with an action group.

◆ PAUSE AND DECIDE

What should the team set up?

A. Daily manual check.
Up to a day late, and depends on one person remembering.

B. Weekly scheduled evaluation.
Detects drift on test data, not live failures within minutes.

C. Agent emails on failure.
Relies on the failing system to report itself, and adds a consequential tool.

D. Log alert with an action group.
Queries live telemetry on a schedule and notifies automatically at the threshold.

THE SIGNAL

'Notify within minutes' when a threshold is crossed: an Azure Monitor alert, not a person or the agent.

Autonomy with safeguards in five lines

- **Autonomy is earned per door.** Scope approvals to consequential tool calls.
- **Approval requests carry server, tool and arguments.** Checkpoint at mandatory gates.
- **No user present:** the agent's own identity, with roles assigned for other resources.
- **Monitor tab, continuous evaluation, red teaming, and log alerts** watch production.
- **Error analysis:** cluster failures, fix the biggest cause, re-evaluate and compare.

> ON THE EXAM
>
> Domain 2 is complete. Next, module 21 begins Domain 3: computer vision.